End-to-End CI/CD Pipeline for a Simple Java Calculator

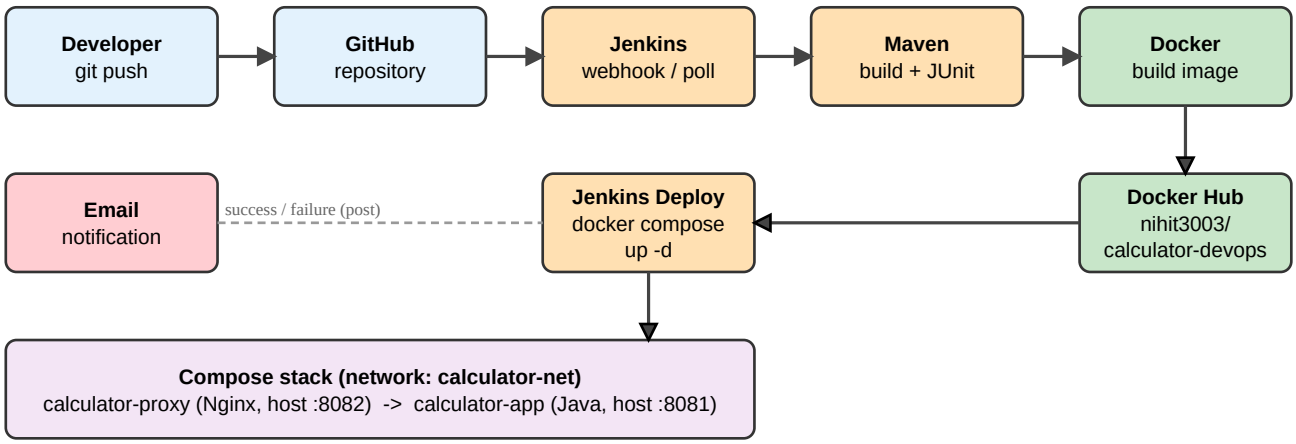
Nihit Reddy Syamala | IMT2023584 | CSE 816 Software Production Engineering, M.Tech (CSE) Term I, 2026-27 | Prof. B. Thangaraju

GitHub: https://github.com/Nihit3003/calculator (replace if different)
Docker Hub: https://hub.docker.com/r/nihit3003/calculator-devops (replace if different)

1. Project summary

A small calculator (add, subtract, multiply, divide) is written in plain Java with Maven and JUnit 5. Following the instructor's direction, it is a simple Java application and does not use Spring Boot. It has a command-line mode and a small server mode built on the JDK's own HTTP server, which provides a real `/health` endpoint (not Spring Actuator) so Docker Compose can run health checks and Nginx has something to proxy. A push to GitHub triggers Jenkins, which builds, tests, packages a Docker image, pushes it to Docker Hub and deploys it with Docker Compose.

2. CI/CD pipeline architecture



Scope is limited to Docker Compose deployment (no Ansible, Kubernetes or Prometheus/Grafana).

3. Pipeline stages

Stage	What it does	Failure behaviour
Checkout	`checkout scm` fetches the latest commit from GitHub, triggered by webhook or Poll SCM.	Pipeline stops.
Build	`mvn -B clean package -DskipTests` compiles and creates `target/calculator.jar`.	Pipeline stops.
Test	`mvn -B test` runs JUnit 5 tests; XML reports are published with the `junit` step.	Failing tests stop the pipeline, so nothing is pushed or deployed.
Docker Build	Multi-stage Dockerfile (Maven+JDK build, JRE-alpine runtime, non-root, HEALTHCHECK). Tagged with the build number and `latest`.	Pipeline stops.
Push	`docker login --password-stdin` using the Jenkins credential `dockerhub-credentials`, then pushes both tags.	Pipeline stops; no secret is stored in Git.
Deploy	`docker compose up -d` with `IMAGE_TAG` = build number, waits for both containers to be healthy, then calls the API through Nginx.	Logs are printed and the build fails.
post	Success and failure emails to the student; `docker image prune` cleanup.	-

Key design points. Testing: 41 JUnit tests cover each operation, divide by zero, missing/invalid input, CLI exit codes and HTTP status codes/JSON. Compose: the app and an Nginx proxy share the user-defined network `calculator-net`, use restart policies and health checks, and Nginx waits for the app (`service_healthy`), while its own check hits `/nginx-health`, which avoids circular dependencies. Idempotence: fixed project and container names plus a unique image tag per build mean re-running the job safely updates the same stack.

4. Challenges faced

[TO BE COMPLETED BY STUDENT: describe the real problems you met while setting this up (for example Jenkins access to Docker, webhook reachability for a local Jenkins, SMTP/App Password setup, port conflicts), how you diagnosed them and how you fixed them. Delete this note before submitting.]

Design challenges that were resolved while preparing the project:

• A plain console program exits immediately, so it cannot be health-checked or proxied. Solved with an optional server mode on the JDK HTTP server.

• Jenkins uses port 8080 by default, so the app is published on 8081 and the proxy on 8082.

• A local Jenkins cannot receive GitHub webhooks, so the Jenkinsfile also includes Poll SCM as a fallback.

5. Evidence (to be attached by student)

[Insert real screenshots here: green Jenkins run with stage view and test results, email notification, `docker ps`, sample API responses, Docker Hub page.]